

形式语言与自动机

Formal Languages and Automata Theory

课程简介及基础知识

计算机科学与技术学院
哈尔滨工业大学（深圳）



郑宜峰

- 授课教师
 - 郑宜峰，博士（后）
 - 研究方向：人工智能安全与隐私、隐私计算等
 - Email: yifeng.zheng@hit.edu.cn
- 研究生助教：
 - 徐双庆，熊天晨
- 考核方法：
 - 平时作业及考核(30%)
 - 期末考试，闭卷(70%)



课程简介

- 培养形式化描述和抽象思维能力，了解和初步掌握“问题—形式化描述—自动化（计算机求解）”这一最典型的计算机问题求解思路。
- 形式语言与自动机理论是计算机科学的基础理论之一，是编译技术的基础，亦在诸如计算机网络协议、文件搜索、数字电路设计和验证等诸多领域发挥着重要作用。通过本课程的学习，为后续相关领域的学习提供理论基础。

- 《Introduction to Automata Theory, Languages, and Computation》
(Third Edition), John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman,
机械工业出版社, 2007.
- 《An Introduction to Formal Languages and Automata》, Peter Linz,
机械工业出版社, 2004.

什么是形式语言与自动机？

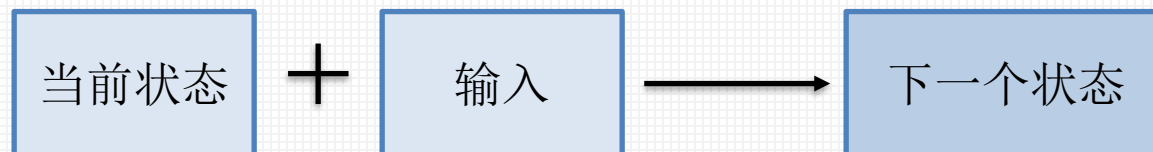
- 形式语言(Formal Languages): 形式化描述的字母表(alphabet)上的字符串(strings)的集合。
 - 字母表是字符的有限集合。
- 例1: 在字母表 $\Sigma = \{a\}$ 上, $L = \{a^{2n+1} | n \geq 0\}$ 表示含有奇数个 a 的符号串的集合。
- 例2: Σ 为C语言所使用的全部符号的集合
 - 正确的C程序是 Σ 上的语言。

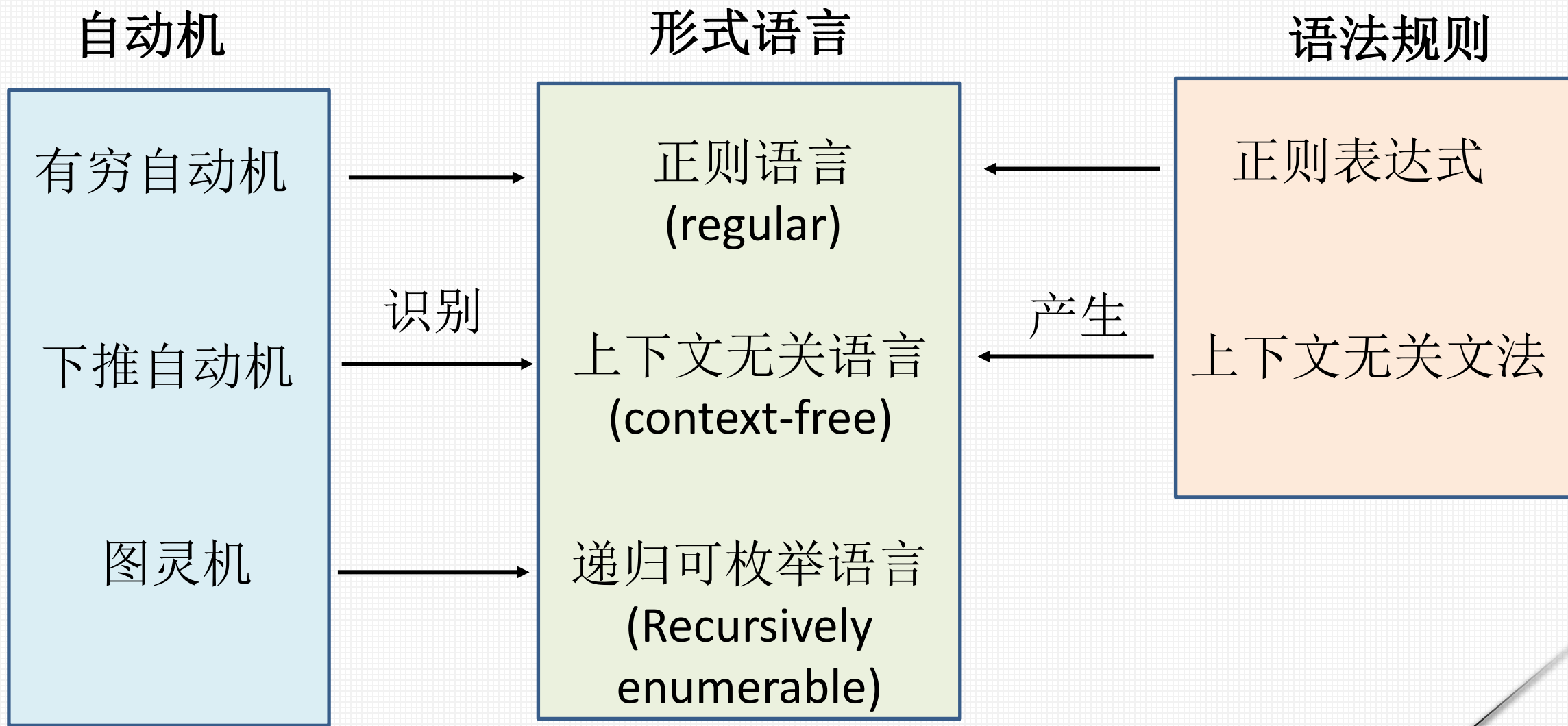
什么是形式语言与自动机？

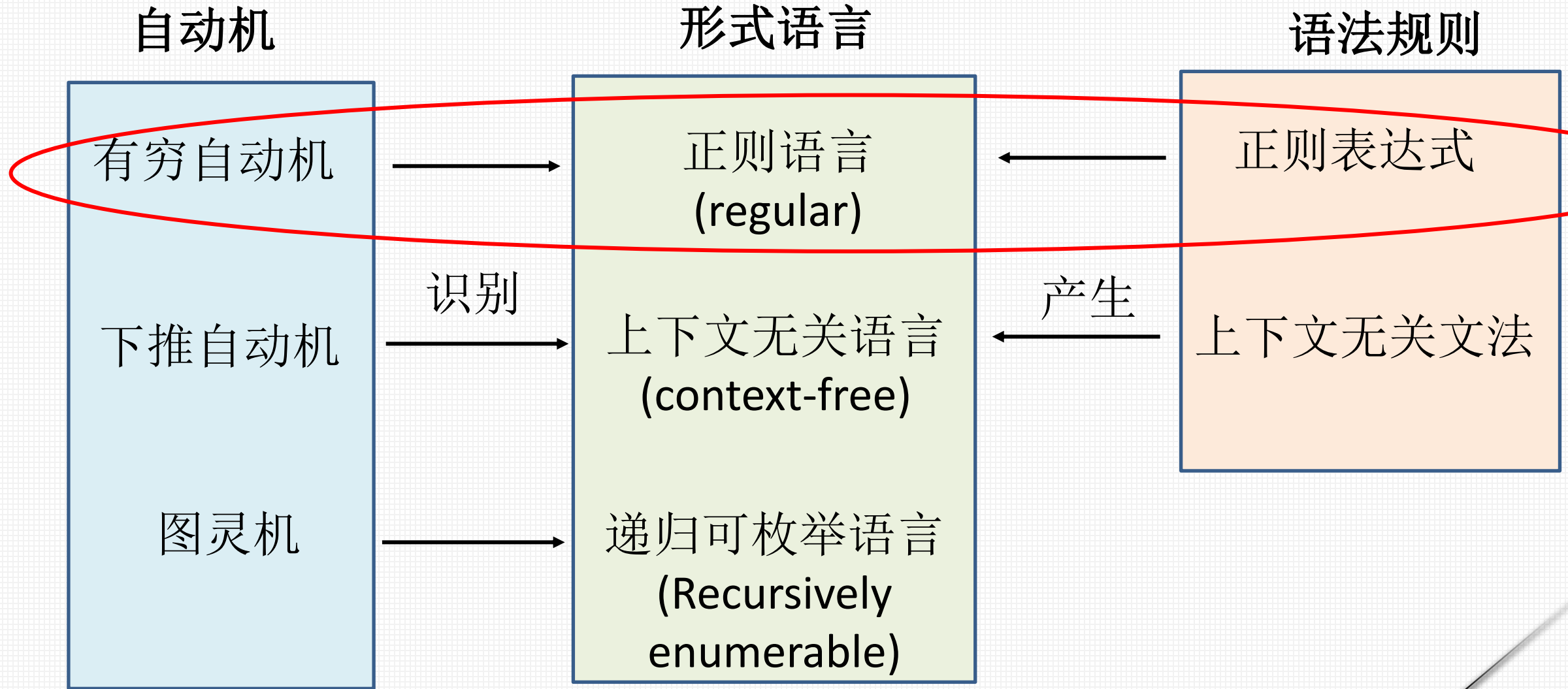
- 形式语言的定义有两种方式：
 - 1. 通过一定的规则(a set of rules)进行描述，根据这些规则能产生相应的符号串
 - 如正则表达式(regular expressions)和上下文无关文法(context-free grammar)

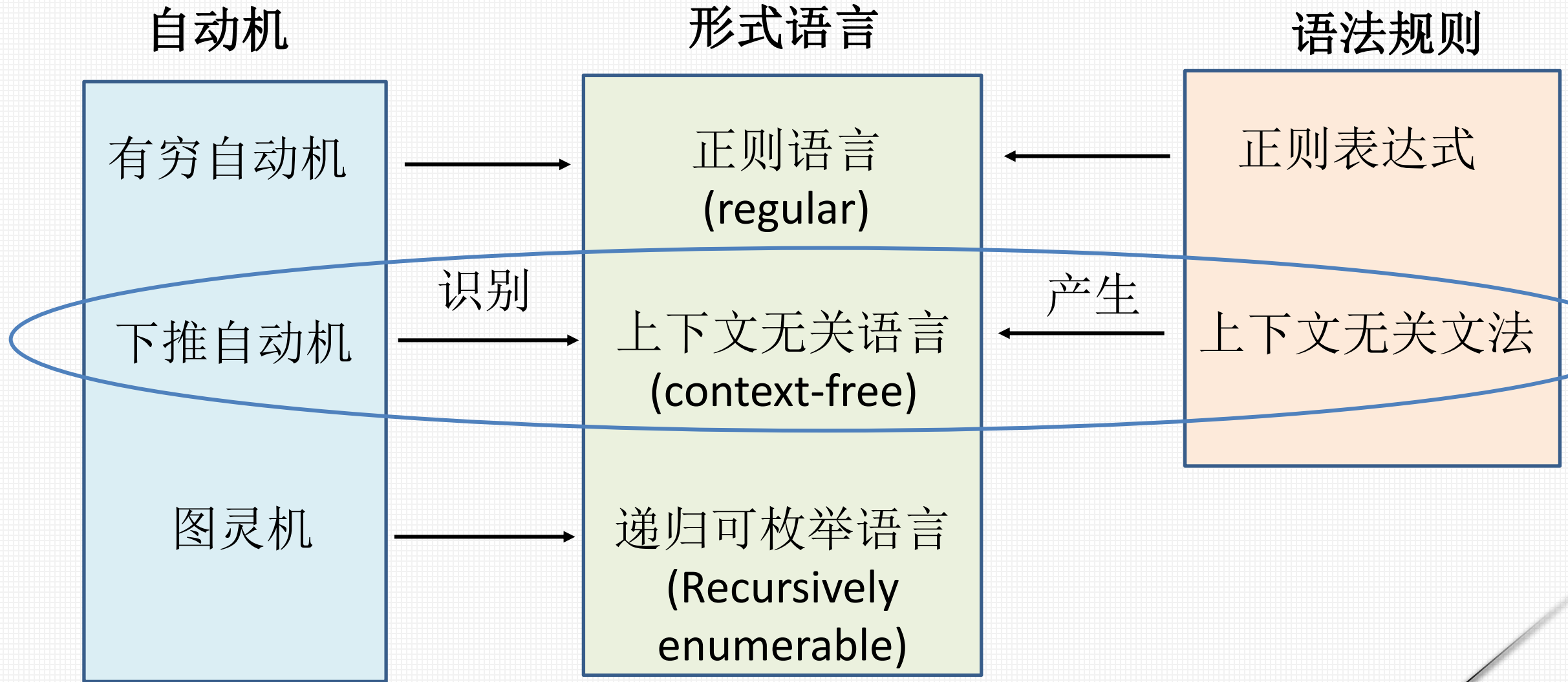
什么是形式语言与自动机？

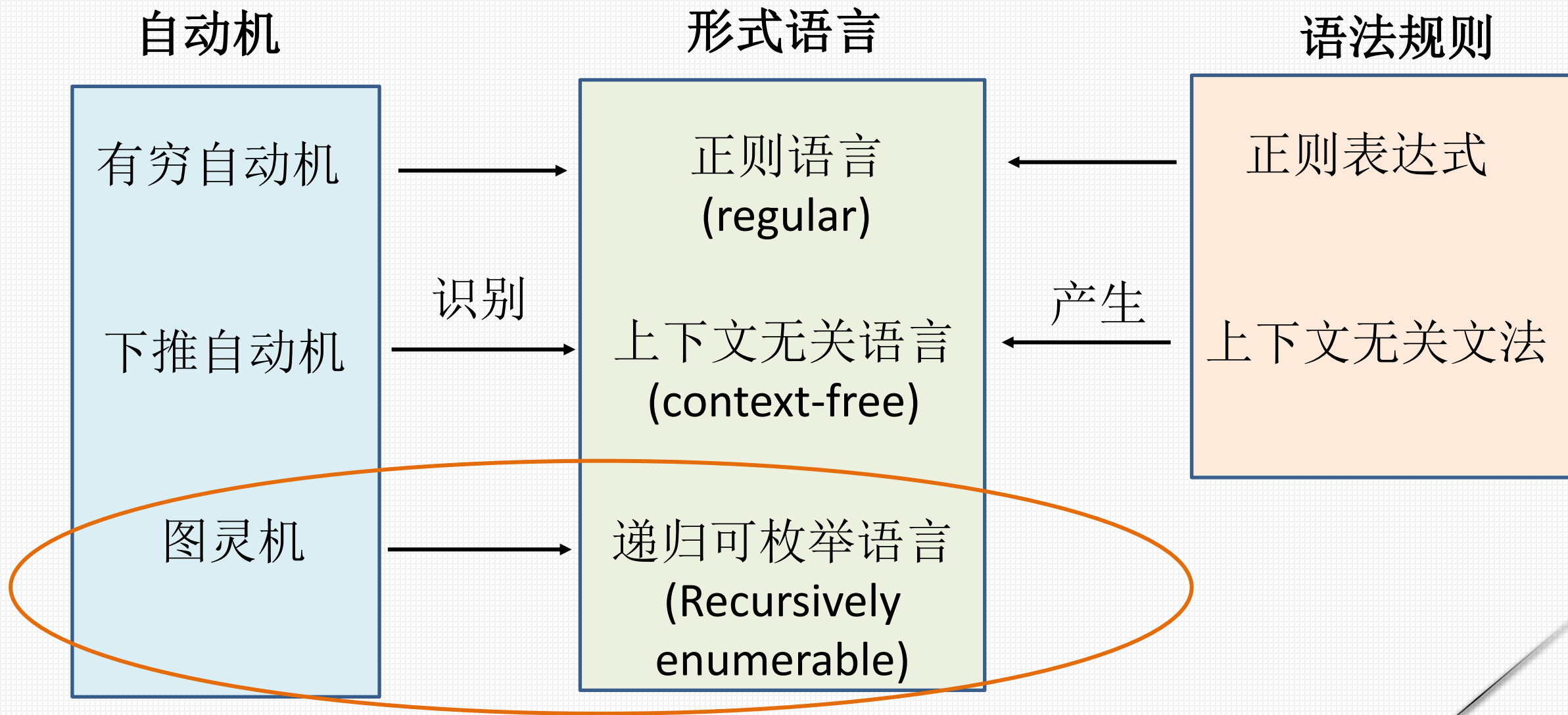
- 形式语言的定义有两种方式：
 - 1. 通过一定的规则(a set of rules)进行描述，根据这些规则能产生相应的符号串
 - 如正则表达式(regular expressions)和上下文无关文法(context-free grammar)
 - 2. 通过抽象化的自动机(automaton)去识别（描述）。
 - 对于按照一定规则构造的自动机（如有穷自动机，下推自动机，图灵机），该自动机就定义了一个形式语言。
 - 该语言由该自动机所能识别的所有**符号串**组成
 - 自动机接受一定的输入，执行一定的动作，产生一定的结果。使用状态迁移描述自动机的整个工作过程。状态区分自动机在不同时刻的状况。
 - 自动机的本质：根据当前状态、输入和规则决定下一个状态。











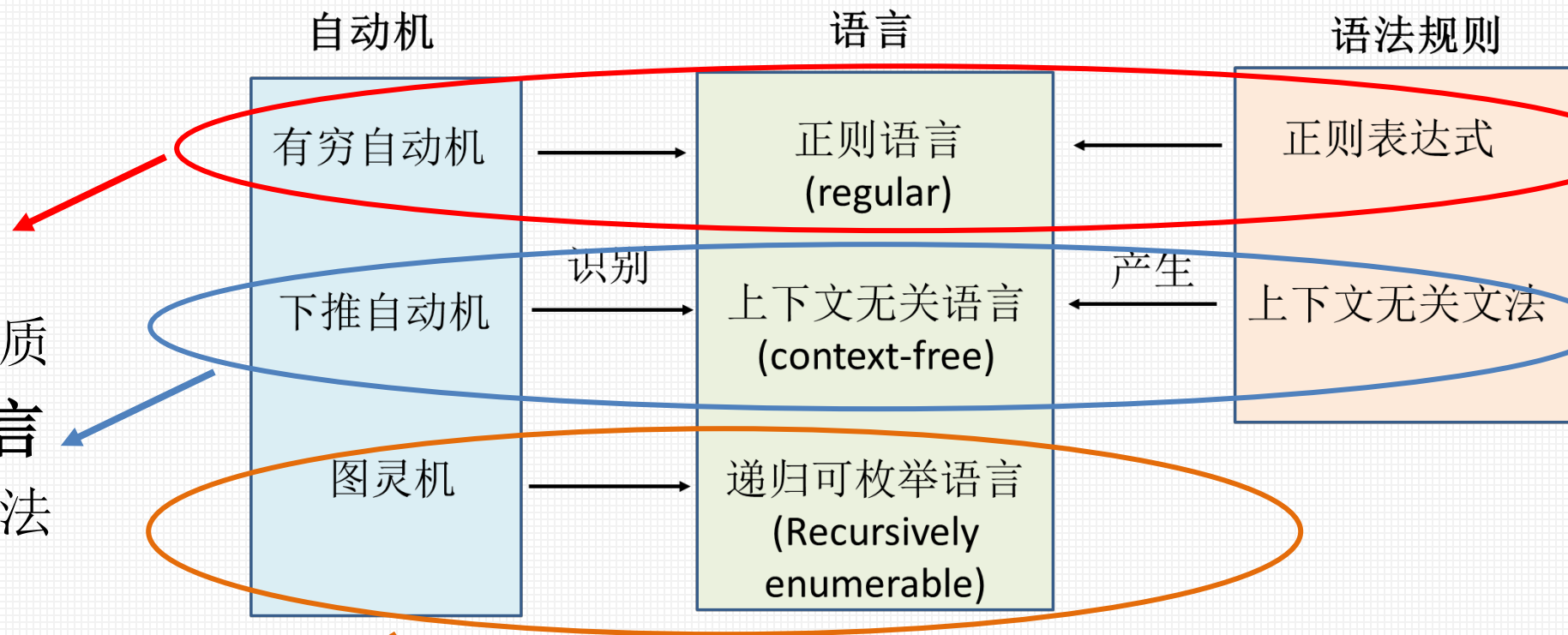
- 正则语言

- 有穷自动机
- 正则表达式
- 正则语言的性质

- 上下文无关语言

- 上下文无关文法
- 下推自动机
- 上下文无关语言的性质

- 图灵机及其扩展简介

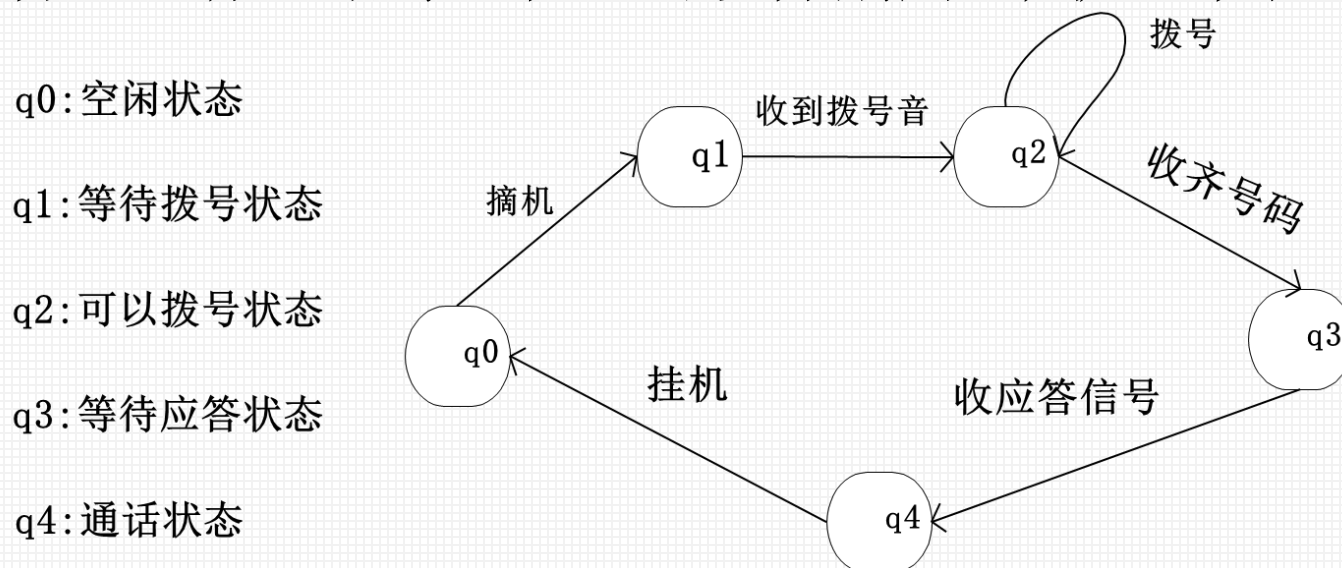


为什么要学习形式语言与自动机？

- 如前所述，形式语言与自动机是编译技术的基础。比如，源程序编译过程中的词法分析会使用到正则表达式、有穷自动机理论，语法结构分析会用到上下文无关文法、下推自动机理论

为什么要学习形式语言与自动机？

- 如前所述，形式语言与自动机是编译技术的基础。比如，源程序编译过程中的词法分析会使用到正则表达式、有穷自动机理论，语法结构分析会用到上下文无关文法、下推自动机理论
- 此外，形式语言与自动机亦在通信、人工智能等领域有着重要应用
 - 通信领域应用示例：在一次呼叫中，从建立连接到通话完毕，要经历摘机，拨号，应答，进行通话等过程，可以分别用5个状态来表示。



为什么要学习形式语言与自动机？

- 从职业角度看，各大厂面试考察内容经常包含形式语言与自动机课程，在各大网站上搜索面经时我们可以发现：

11月中旬，一面面试官加我微信，发起面试

一面：不得不说鹅厂的考察范围真的很广，面了一个多小时，涉及计算机的方方面面

1. TCP/IP协议
2. 什么是UDP，跟TCP的关系，啥时候用，能不能既要UDP的速度也要TCP的可靠性，光这个讨论了5分钟，循环冗余校验，滑动窗口等等TCP的特性都说了遍
3. 两个栈实现队列
4. 操作系统，虚拟内存
5. C++特性，RTTI，右值左值，引用指针，STL，红黑树，hash
6. **形式语言与自动机**，问了个字符串匹配，说到正则表达式，问我做过，扯了下
7. 网易实习的经历
8. 如何实现一个实时的游戏积分表，树状数组
应该还有些别的，总之觉得鹅厂考察范围是真的广

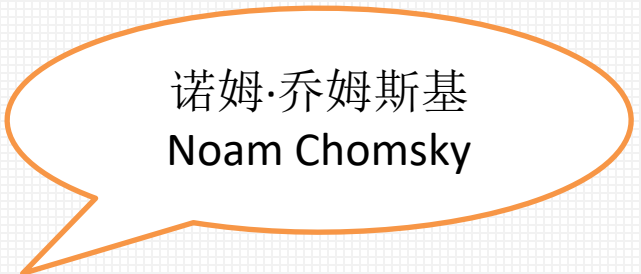


秋招大厂算法岗面经与提问总结（蚂蚁/美图/宜信/滴滴）

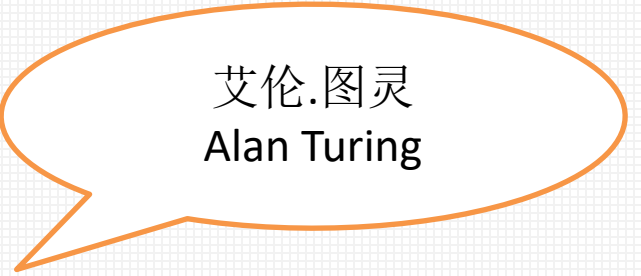


- 验证码图片的去噪和提取字符
- **有限状态自动机**，然后要我画状态转移图。
- 聚类
- 用过哪些聚类算法，解释密度聚类算法。

- 正则语言
 - 有穷自动机
 - 正则表达式
 - 正则语言的性质
- 上下文无关语言
 - 上下文无关文法
 - 下推自动机
 - 上下文无关语言的性质
- 图灵机及其扩展简介



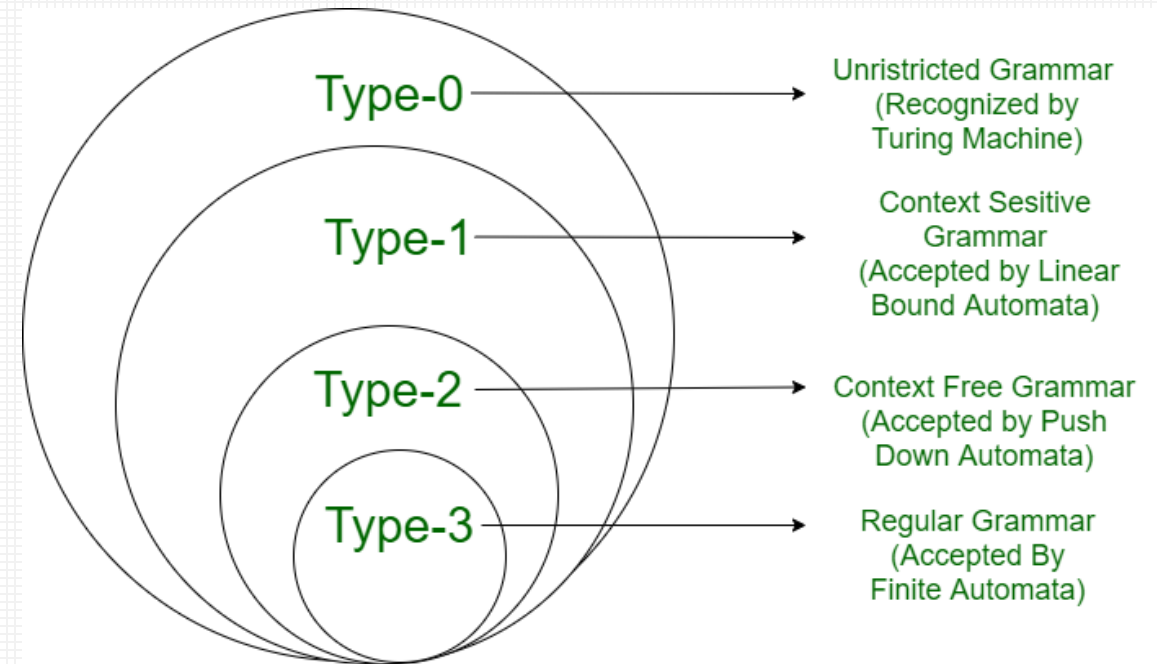
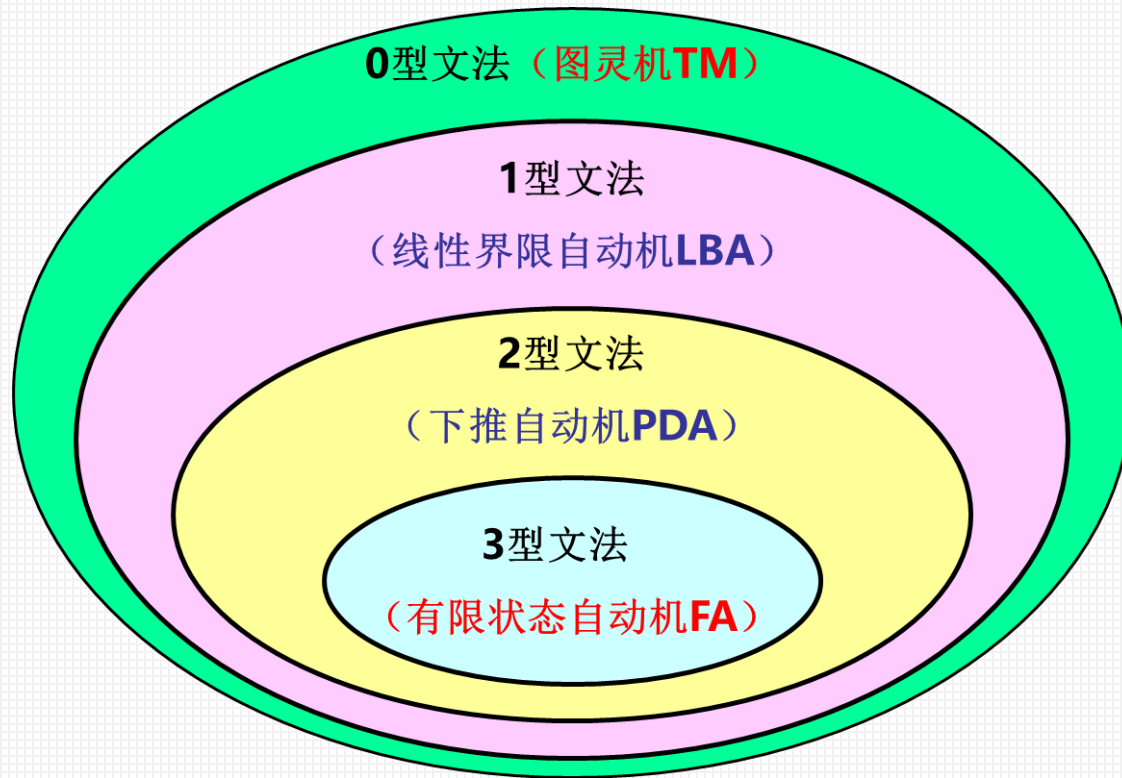
诺姆·乔姆斯基
Noam Chomsky



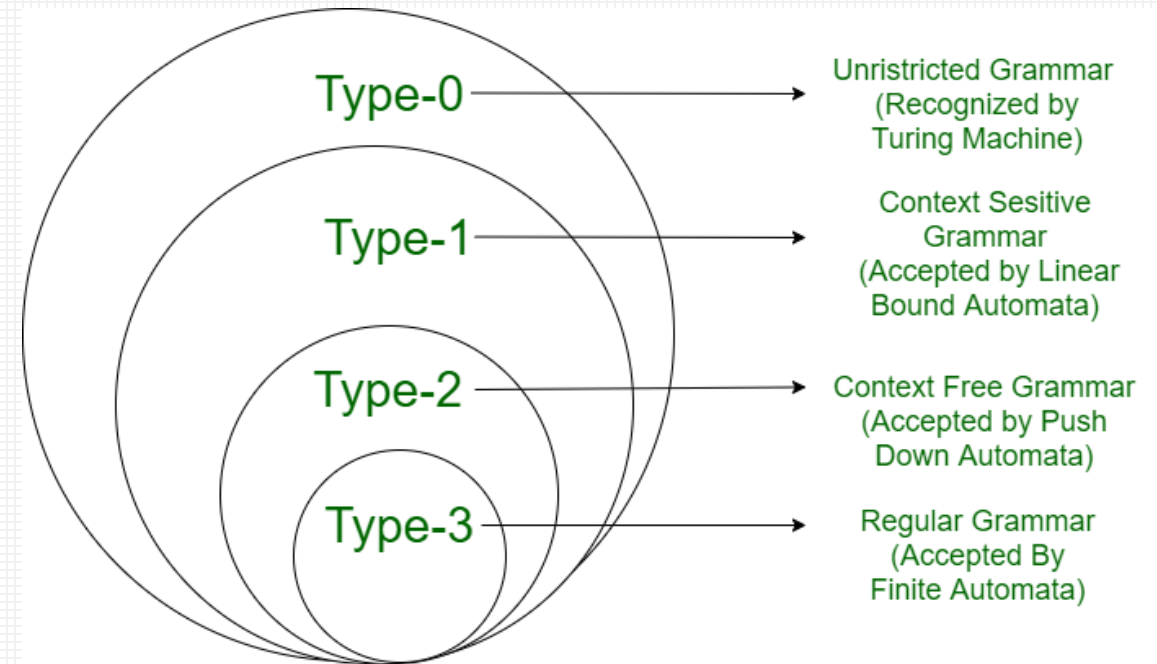
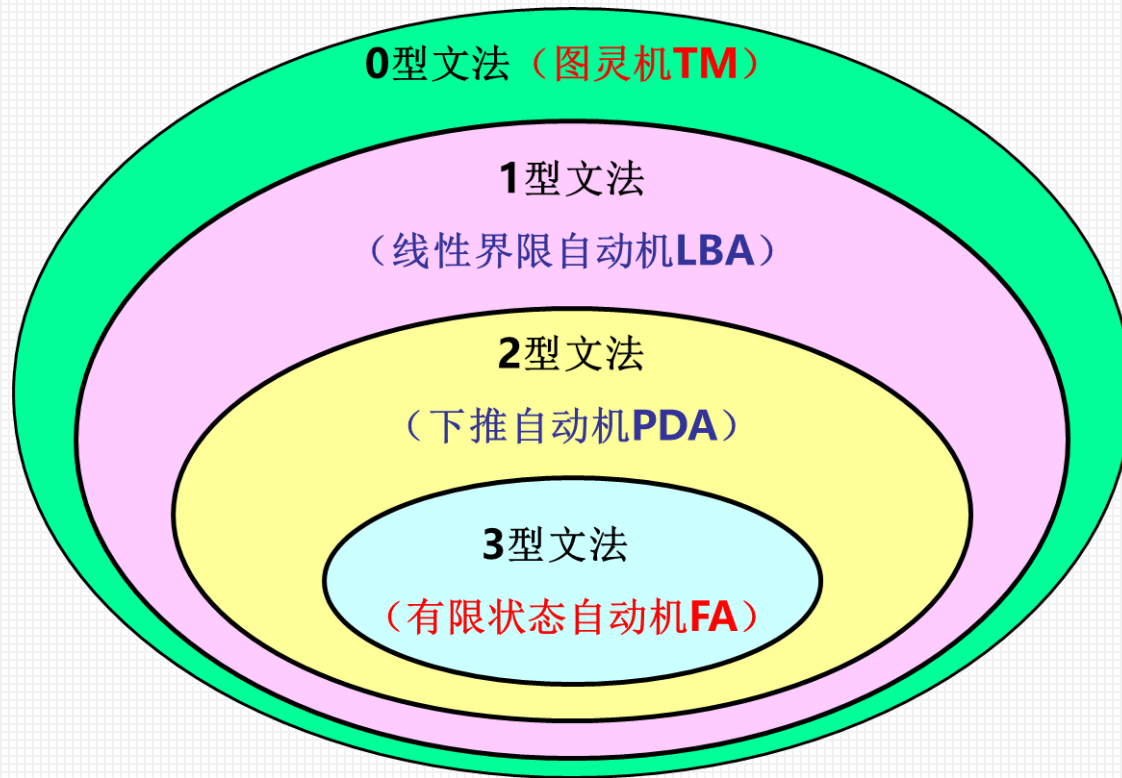
艾伦·图灵
Alan Turing

- ▶ 诺姆·乔姆斯基（1928年12月7日—）是美国语言学家、哲学家、认知科学家、逻辑学家、政治评论员和活动家。目前是MIT的名誉教授。
- ▶ 乔姆斯基提出了乔姆斯基层级 (Chomsky Hierarchy): 根据文法生成能力不同而对描述形式语言的文法进行分类。这是计算机科学中刻画形式文法表达能力的一个分类谱系。





- 从0型到3型文法，文法规则受到的限制变多，生成语言的能力变弱（即可生成的语言的复杂性变低）。
 - 无限制文法->上下文有关文法->上下文无关文法->正则文法
 - 递归可枚举语言->上下文有关语言-上下文无关语言-正则语言



- 从0型到3型文法，文法规则受到的限制变多，生成语言的能力变弱（即可生成的语言的复杂性变低）。
 - 无限制文法->上下文有关文法->上下文无关文法->正则文法
 - 递归可枚举语言->上下文有关语言->上下文无关语言->正则语言

- 图灵在计算机科学的发展中有着巨大的影响力，他用图灵机对“算法”和“计算”的概念进行了形式化表示，图灵机通常被视为通用计算机的模型。
- 图灵常被认为是计算机科学和人工智能之父。



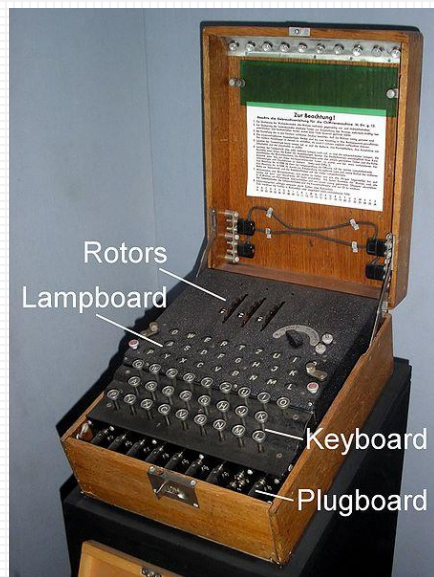
Alan Mathison Turing
(23 June 1912 – 7 June 1954)

- 图灵于1912年6月23日生于英国伦敦。少年时就表现出独特的直觉创造能力和对数学的爱好。
- 1927年末，年仅15岁的图灵开始阅读爱因斯坦的著作，表现出他已具备非同凡响的数学水平和科学理解力。他不仅掌握了这一点，而且还从一段从未明确说明的文本中推断出爱因斯坦对牛顿运动定律的质疑。



图灵在二战中的工作

- 第二次世界大战打断了图灵的正常研究工作，1939年秋，他应召到英国外交部通信处从事军事工作，主要是破译敌方密码的工作。由于破译工作的需要，他参与了世界上最早的电子计算机的研制工作。他的工作取得了极好的成就，因而于1945年获政府的最高奖——大英帝国荣誉勋章（O.B.E.勋章）。

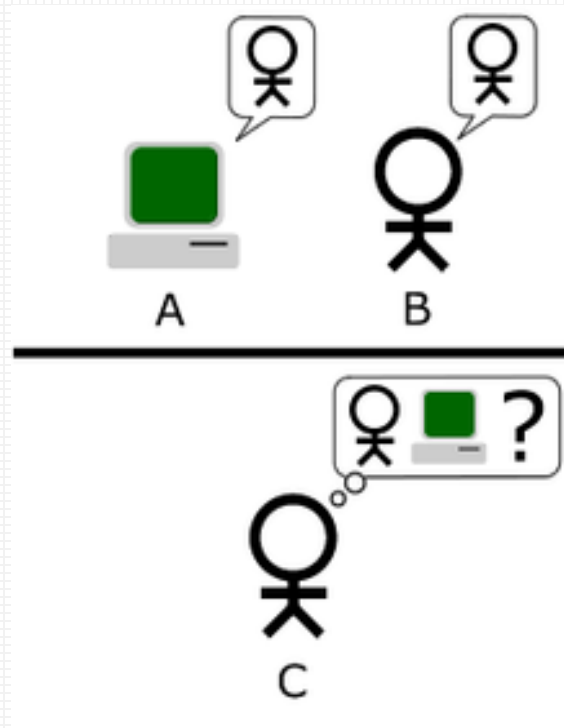


Enigma machine



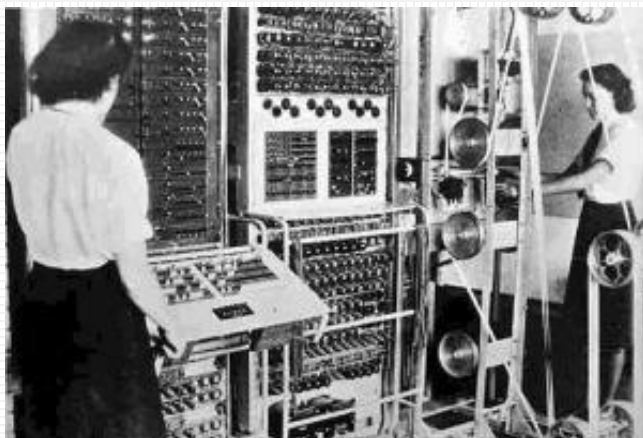
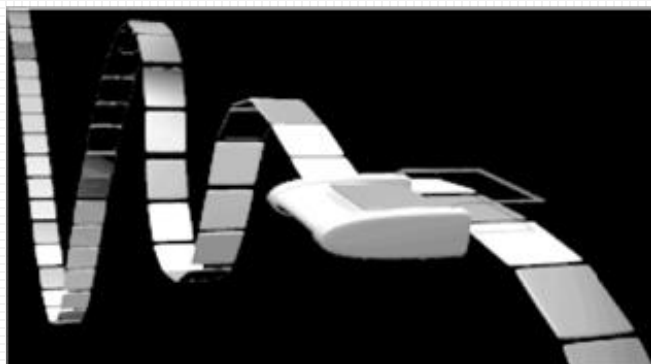
Turing at Bletchley Park

- 图灵写了一篇题为“机器能思考吗？”的文章，提出了一个被称为图灵测试的实验，试图为机器定义一个称为“智能”的标准。



图灵机 Turing machine

- 1936年，图灵提出了一种抽象的计算模型——图灵机。
- 图灵机是一种假想的设备，它根据规则表操纵纸袋上的符号，并进行状态之间的转移。
- 这个设想最有变革意义的地方在于，它第一次在纯数学的符号逻辑和实体世界之间建立了联系，后来我们所熟知的计算机，以及还没有实现的“人工智能”，都基于这个设想。



图灵奖 Turing Award

1996年	阿米尔·伯努利	Amir Pnueli	时序逻辑，程序与系统验证
1997年	道格拉斯·恩格尔巴特	Douglas Engelbart	互动计算
1998年	詹姆斯·尼古拉·格雷	James Gray	数据库与事务处理
1999年	弗雷德里克·布鲁克斯	Frederick Phillips Brooks, Jr.	计算机体系结构，操作系统，软件工程
2000年	姚期智	Andrew Chi-Chih Yao	计算理论，包括伪随机数生成，密码学与通信复杂度
2001年	约翰·达尔	Ole-Johan Dahl	面向对象编程
	卡琳·奈加特	Kristen Nygaard	
	罗纳德·李维斯特	Ronald L. Rivest	
2002年	阿迪·沙米尔	Adi Shamir	公钥密码学 (RSA加密算法)
	莱昂纳德·阿德勒曼	Leonard M. Adleman	
2003年	艾伦·凯	Alan Kay	面向对象编程
2004年	文顿·G· Cerf	Vinton G. Cerf	TCP/IP协议
	罗伯特·E· Kahn	Robert E. Kahn	
2005年	彼得·诺尔	Peter Naur	Algol 60语言
2006年	弗朗西斯·E· 艾伦	Frances E. Allen	优化编译器
2007年	爱德蒙·M· 克拉克	Edmund M. Clarke	开发自动化方法检测计算机硬件和软件中的设计错误
	艾伦·爱默生	Allen Emerson	
	约瑟夫·Sifakis	Joseph Sifakis	
2008年	芭芭拉·Liskov	Barbara Liskov	编程语言和系统设计的实践与理论
2009年	查尔斯·萨克尔	Charles Thacker	帮助设计、制造第一款现代PC
2010年	莱斯利·瓦伦特	Leslie Valiant	对众多计算理论所做的变革性的贡献

2000年	姚期智	Andrew Chi-Chih Yao	计算理论，包括伪随机数生成，密码学与通信复杂度
-------	-----	---------------------	-------------------------

ACM A.M Turing Award，是由美国计算机协会（ACM）于1966年设立的计算机奖项，名称取自艾伦·麦席森·图灵

（Alan M. Turing），旨在奖励对计算机事业作出重要贡献的个人。

图灵奖对获奖条件要求极高，评奖程序极严，一般每年仅授予一名计算机科学家。图灵奖是计算机领域的国际最高奖项，被誉为“计算机界的诺贝尔奖”。

基础知识

- 字母表(alphabet):符号（字符）的**非空有穷**集
 - 例:

$$\Sigma_1 = \{0,1\}$$

$$\Sigma_2 = \{a, b, \dots, z\}$$

$$\Sigma_3 = \{a, b\}$$

- **字符串(string)**:由某字母表中符号组成的有穷序列, 又称为句子。
 - 若 $\Sigma_1 = \{0,1\}$, 那么 $0,1,00,111001$ 为 Σ_1 上的字符串。
 - 若 $\Sigma_2 = \{a,b,\dots,z\}$, 那么 ab,bcd 为 Σ_2 上的字符串。
- **空串(empty string)**:记为 ε (epsilon), 有0个字符的串。
- 符号使用的一般约定:
 - 字母表: $\Sigma, \Gamma, \dots$
 - 字符: $a, b, c, \dots$
 - 字符串: $\dots, w, x, y, z,$
 - 集合: $A, B, C, \dots$

- **字符串的长度**: 字符串中符号所占位置的个数, 记为 $|\cdot|$ 。
- 若字母表为 Σ , 可递归定义为:

$$|w| = \begin{cases} 0 & , w = \varepsilon \\ |x| + 1 & , w = xa \end{cases}$$

其中 $a \in \Sigma$, w 和 x 是 Σ 中字符组成的字符串。

- **字符串 x 和 y 的连接**: 将首尾相接得到新字符串的运算, 记为 $x \cdot y$ 或 xy 。
- 同样, 可递归定义为:

$$xy = \begin{cases} x & , y = \varepsilon \\ (xz)a, & y = za \end{cases}$$

其中 $a \in \Sigma$, 且 x, y, z 都是字符串。

- 对任何字符串 x , 有 $\varepsilon \cdot x = x \cdot \varepsilon = x$ 。

- 字符串 x 的 n 次幂 ($n \geq 0$) , 递归定义为

$$x^n = \begin{cases} \varepsilon, & n = 0 \\ x^{n-1}x, & n > 0 \end{cases}$$

例如, 若 $\Sigma = \{a, b\}$, 那么

$$(ba)^2 = (ba)^1ba = (ba)^0baba = \varepsilon baba = baba$$

$$ba^2 = ba^1a = ba^0aa = b\varepsilon aa = baa$$

- 集合:某一类对象形成的整体叫做**集合(set)**。集合的成员称之为**元素**。
- 子集: 如果集合 A 中的元素都是集合 B 的元素, 则集合 A 是集合 B 的子集(subset), 记作 $A \subseteq B$ 。如果 $A \subseteq B$, 存在元素 $x \in B$ 且 $x \notin A$, 则 A 是 B 的真子集, 记作 $A \subset B$
 - 空集是任意集合的子集, 是任意非空集合的真子集。

- 集合:某一类对象形成的整体叫做**集合(set)**。集合的成员称之为**元素**。
- 子集: 如果集合 A 中的元素都是集合 B 的元素, 则集合 A 是集合 B 的子集(subset), 记作 $A \subseteq B$ 。如果 $A \subseteq B$, 存在元素 $x \in B$ 且 $x \notin A$, 则 A 是 B 的真子集, 记作 $A \subset B$
 - 空集是任意集合的子集, 是任意非空集合的真子集。
- 幂集(power set):集合 A 的所有子集 (包括空集) 组成的集合, 称为 A 的幂集, 记作 2^A
 - 例: $A = \{4,7\}, 2^A = \{\{ \} \text{ or } \emptyset, \{4\}, \{7\}, \{4,7\}\}$

- 集合的基本运算：交(intersection)、并(union)、补(complement)
 - 交： $A = \{1,2,3\}, B = \{2,3\}, A \cap B = \{2,3\}$
 - 并： $A = \{1,2,3\}, B = \{2,3\}, A \cup B = \{1,2,3\}$
 - 补运算: 设 S 是一个集合， A 是集合 S 的子集，则 A 的补集由 S 中所有不属于 A 的元素组成，记作 A^C 。
- 集合之间的差(set difference): 属于 A ，但不属于 B 的所有元素组成的集合叫做 A 与 B 的差，记作 $A - B = \{a | a \in A \text{ 且 } a \notin B\}$
 - 例： $A = \{1,2,3,4\}, B = \{3,4,5,6\}, A - B = \{1,2\}$

- 集合 A 和 B 的笛卡尔积(Cartesian Product): 集合 A 与 B 的笛卡儿积是一个集合, 该集合是由所有这样的有序对 (a, b) 组成的: 其中 $a \in A$, $b \in B$, 记作 $A \times B$
 - $\{3,5\} \times \{6,7\} = \{(3,6), (3,7), (5,6), (5,7)\}$
- 集合 A 和 B 的连接(concatenation or product), 记为 AB , 定义为
$$AB = \{w | w = xy, x \in A \text{ 且 } y \in B\}$$
 - 例: $A = \{a, aa\}, B = \{a, ba\}, AB = \{aa, aba, aaa, aaba\}$

- 集合 A 的 n 次幂 ($n \geq 0$) , 递归定义为

$$A^n = \begin{cases} \{\varepsilon\}, & n = 0 \\ A^{n-1}A, & n \geq 1 \end{cases}$$

那么, 若 Σ 为字母表, 则 Σ^n 为 Σ 上长度为 n 的字符串的集合。

如果 $\Sigma = \{0,1\}$,有

$$\Sigma^0 = \{\varepsilon\}$$

$$\Sigma^1 = \{0, 1\}$$

$$\Sigma^2 = \{00, 01, 10, 11\}$$

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

...

- 克林闭包(Kleene Closure):

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$$

即 Σ^* 为 Σ 上所有可能的字符串的集合（含有空串）

- 正闭包(Positive Closure):

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$$

即 Σ^+ 为 Σ 上除去空串后所有可能的字符串的集合（不含有空串）

显然, $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ 。

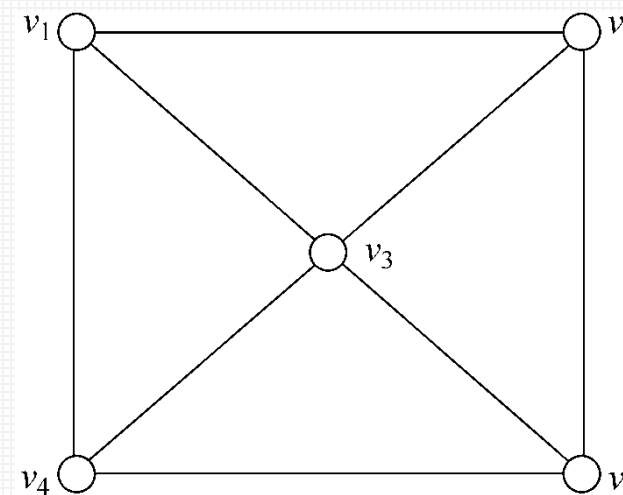
- 定义:若 Σ 为字母表, 则任何集合 $L \subseteq \Sigma^*$ 是字母表 Σ 上的一个语言。
- 例: 设字母表 $\Sigma = \{0,1\}$, 则
 - $L_1 = \{0^n 1^n | n \geq 0\}$
 - $L_2 = \{0^{2n+1} | n \geq 0\}$
 - $L_3 = \{\varepsilon\}$
 - $L_4 = \emptyset$均是字母表 Σ 上的语言。

- 自动机理论中的典型问题：判定给定的字符串 w 是否属于某个具体的语言 L :

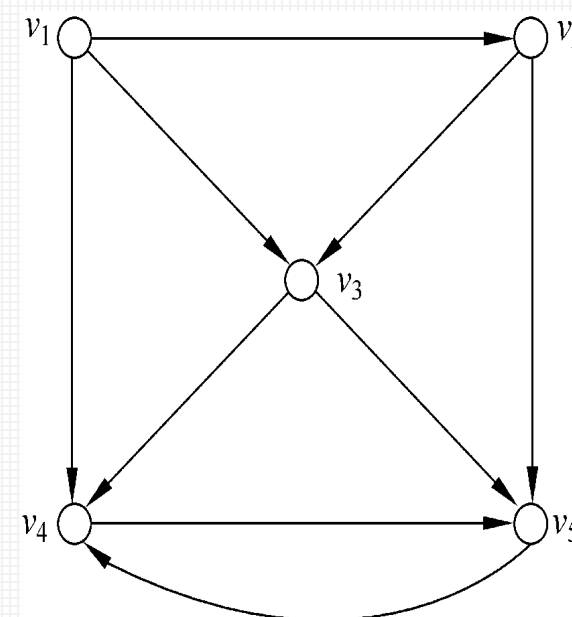
$$w \in L?$$

成员性(membership)问题是自动机理论的核心问题。

- 图由顶点(vertex)和边(edge)组成
- 无向图:
 - 顶点的度: 和该顶点相关联的边数。 $\deg(v)$
- 有向图:
 - 顶点的入度(indegree): 指向 (进入) 该顶点的边数。 $\deg^-(v)$
 - 顶点的出度(outdegree): 从该顶点出发的边数. $\deg^+(v)$

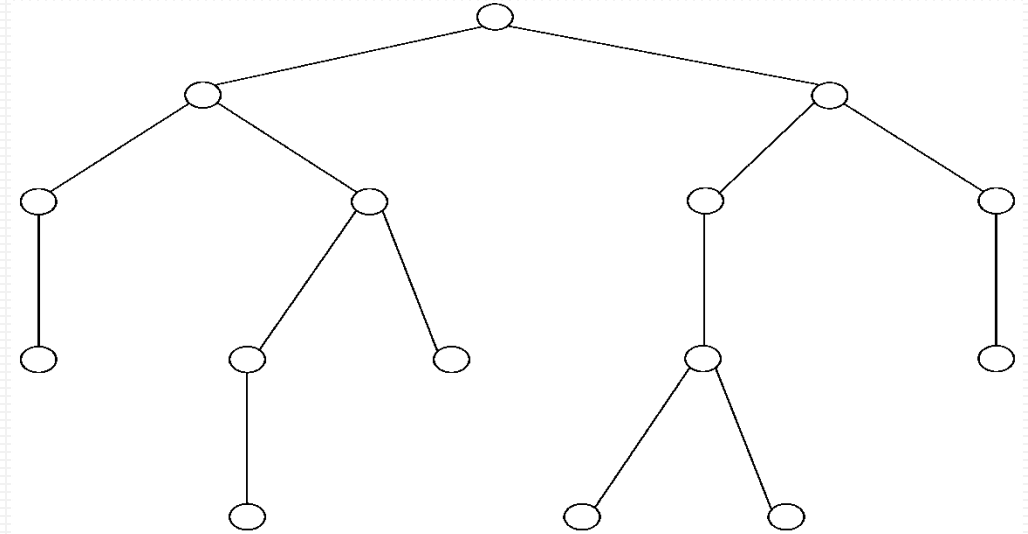


$\deg(v_1)=3,$
 $\deg(v_2)=3,$
 $\deg(v_3)=4,$
 $\deg(v_4)=3,$
 $\deg(v_5)=3$



$\deg^-(v_1)=0,$
 $\deg^+(v_1)=3,$
 $\deg^-(v_2)=1,$
 $\deg^+(v_2)=2,$
...

- 二叉树 (binary tree) :
 - 每个节点最多只有两个子节点, 没有子节点的称为叶子节点
 - 第一层为根节点(root node), 第 i 层($i \geq 1$)最多只有 2^{i-1} 个节点
 - 一棵 n 层二元树最多有个 2^{n-1} 叶子节点。



- 演绎证明 (Deductive Reasoning)
 - 通常来讲，是从一般性原理出发，依据已被确认的事实和公认的逻辑规则，推导出某个特殊情况下的结论
 - “三段论”是演绎证明的一般模式，包括：
 - 大前提：已知的一般性原理
 - 小前提：所证明对象的特殊性
 - 结论：根据一般性原理，对特殊情况作出判断
- 例：所有奇数都不能被2整除(大前提)， $2100+1$ 是奇数(小前提)，则 $2100+1$ 不能被2整除(结论)

- 归纳证明(Induction)

- 从某类事物的部分对象具有某些特征，推导出该类事物的全部对象都具有这些特征的过程，是由个别事实概括出一般结论的过程
- 是处理递归定义的对象必不可少的证明方法
- 由“基础”和“归纳递推”两部分组成，实际上处理的递归定义对象也通常包含这两部分
- 递归定义
 - 基础(basis): 基本的元素定义
 - 归纳(induction): 指出用已有元素来构造新元素的规则

- 例:若 x 和 y 是 Σ 上的字符串, 请证明 $|xy| = |x| + |y|$ 。
- 证明:通过对 $|y|$ 的归纳来证明。

(1) 基础:当 $|y| = 0$,即 $y = \varepsilon$

$$|x\varepsilon| = |x|$$

连接的定义

$$= |x| + |\varepsilon|$$

长度的定义

(2)归纳:假设 $|y| = n$ 时命题成立,

则当 $|y| = n + 1$, 即 $y = wa$

$$|xy| = |(xw)a|$$

连接的定义

$$= |xw| + 1$$

长度的定义

$$= |x| + |w| + 1$$

归纳假设

$$= |x| + |wa|$$

长度的定义

长度定义:

$$|w| = \begin{cases} 0 & , w = \varepsilon \\ |x| + 1, w = xa \end{cases}$$

- 反证法(**proof by contradiction**): 首先假设反命题成立（即在原命题的条件下, 结论不成立）, 然后推理出明显矛盾的结果, 从而判定该假设实际上不成立, 因此原命题得证。
- 一般形式, 将证明命题“如果 H , 则 C ”转化成证明命题“ H 与非 C 出现矛盾”
 - 例, H = “ U 是无穷集, S 是 U 的有穷子集, T 是 S 关于 U 的补集”, 则 C = “ T 是无穷的”
 - 证明: 假设“非 C ”, 即 T 是有穷的。因为 S 是有穷的, $U = T \cup S$, 则 U 是有穷集, 与 H 中“ U 是无穷集”矛盾。故假设“非 C ”不成立, C 得证。

- 反例证明：通过举出一个例子，使之具备命题的条件，而不具有命题的结论，来证明命题是假的。这种例子称为反例。
 - 例，所有素数都是奇数。更加形式化的描述是：“如果 x 是素数，则 x 是奇数”
 - 否定：整数2是素数，但2是偶数

- 课程简介
- 基本概念
 - 字母表、字符串、集合、克林闭包、图、二叉树
- 基本的数学证明方法
 - 演绎法、归纳法、反证法